QwikVote

Gavin D’Hondt, Ricky Hodge, Adam Walsh

CSI 4160

Problem Statement

- Group decisions are slow in async chat (time, idea, tool, naming decisions)
- Existing tools are either too basic (simple vote counting) or too heavy (survey platforms)
- They miss key consensus dynamics: vote intensity, veto power, fair tie-breaking
- Polls are used everywhere by everyone today, and yet the existing platforms for polls are lackluster, due to the perceived simplicity of polls

Goal: reduce decision friction and resolve group choices faster and more fairly

Proposed Solution

- QwikVote is a zero-friction web polling platform for fast group consensus
- Help people overcome analysis paralysis, saving their limited time and resolving conflict quicker and easier
- Weighted voting, veto support, and random tie-break for fairness
- LLM-assisted option generation to reduce poll setup effort
- Simplifying this process at a granular level has the potential to massively scale up and have exponentially beneficial effects on individuals.

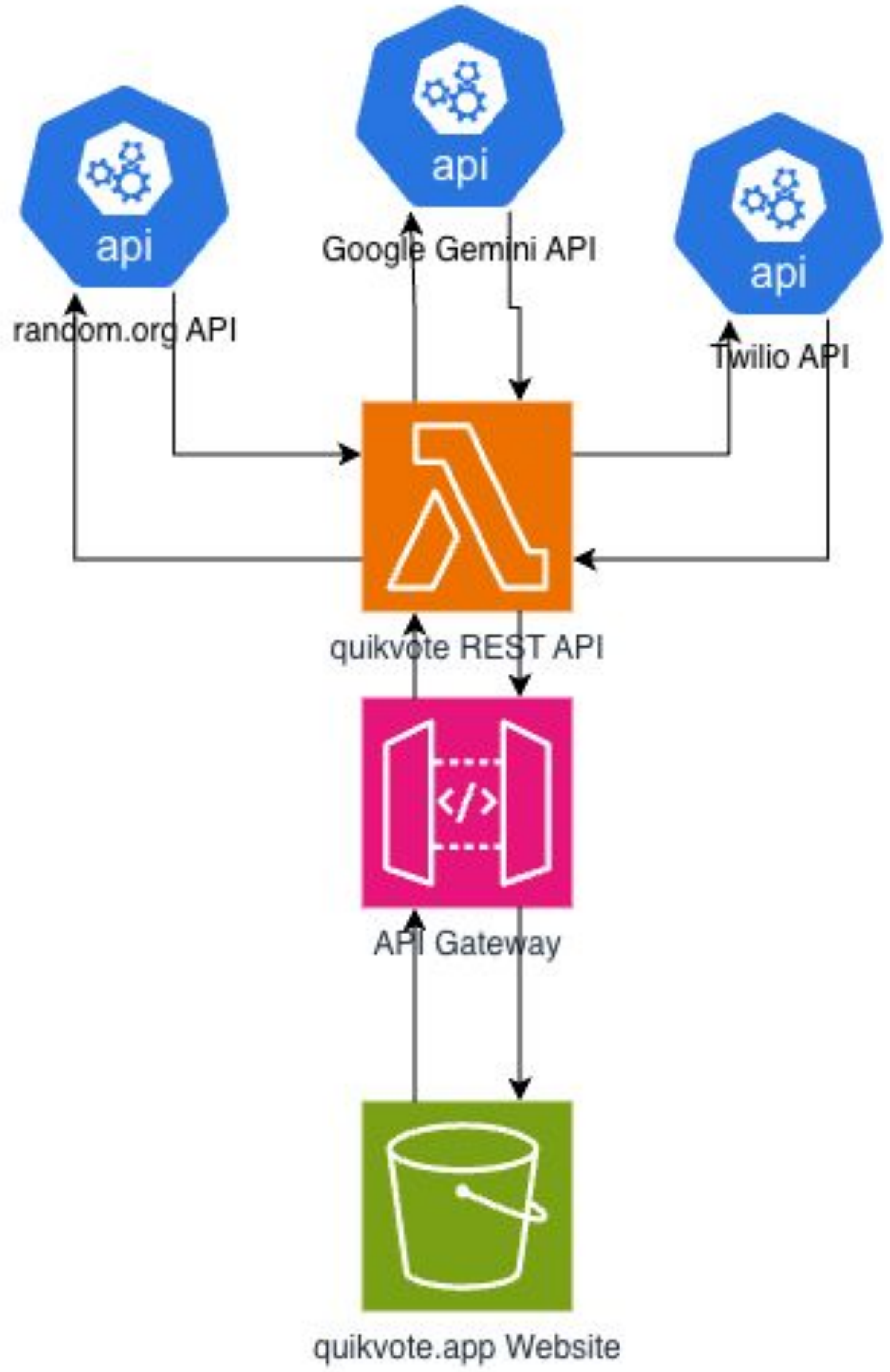
Related Work

- Freixas and Pons (2021) examined the mathematical properties of anonymous and weighted voting systems, demonstrating the complexity of designing fair aggregation mechanisms when voters have multiple options and outcomes are multidimensional.
- Huang et al. (2024) developed a collaborative group decision support system using multi-actor, multicriteria analysis (MAMCA), demonstrating how structured approaches can hugely improve group decision outcomes.

Gap: research exists, but casual tools rarely implement these mechanisms well

System Architecture

- **Frontend**: React + TypeScript + Vite + TanStack SPA
- **VotingService**: Python + Magnum REST/BFF API with poll + vote business logic on Lambda w/ API GW
- **SuggestionService**: Gemini-based option generation
- **RandomnessService**: Random.org tie-break support
- **NotificationService**: Twilio + SendGrid
- **Cloud deployment**: AWS + CDK for scalable, modular microservices



API Endpoints

POST /polls

- **Input**: Poll Creation Request structure
- **Output**: Poll Object including a generated poll_id

POST /polls/{poll_id}/vote

- **Input**: Vote Submission Request structure
- **Output**: Confirmation of vote submission and updated aggregated scores

GET /polls/{poll_id}

- **Input**: Request from client
- **Output**: Poll Object and optionally a Poll Results Object

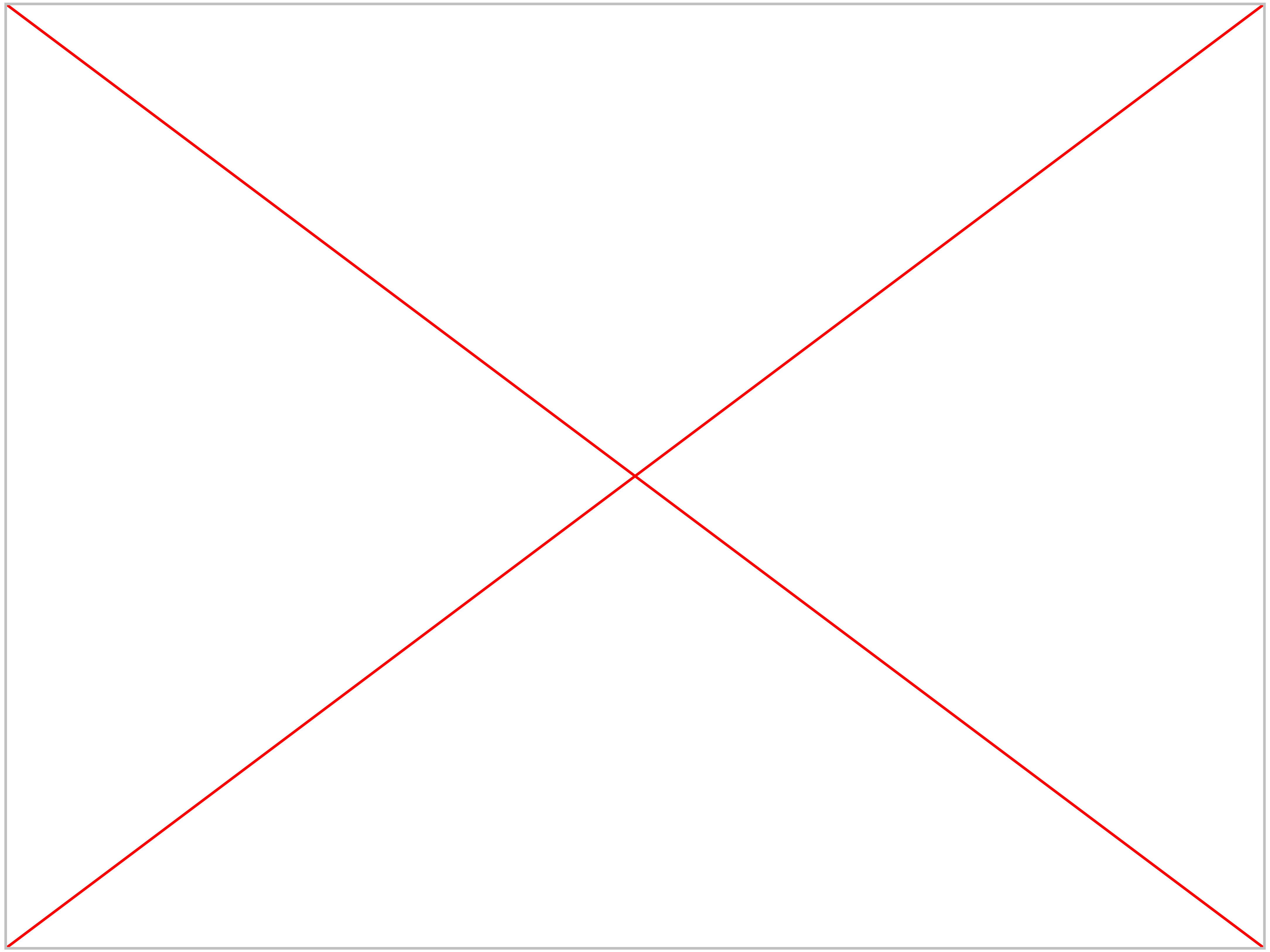
POST /polls/{poll_id}/close

- **Input**: Close request from client
- **Output**: Finalized Poll Results Object including the winning option and justification

Methodology/Workflow

- **Request Initiation**: Client application sends a request to either make a poll, submit a vote, or retrieve results
- **Poll Creation and LLM Option Generation**: API generates a prompt which is sent to LLM to generate option suggestions
- **Vote Processing and Reconciliation**: validates poll status, applies weighted scoring and vetos if enabled, updates total votes in database
- **Tie Breaking**: After applying weighted score and veto logic, breaks any tie using random.org and generates justification on how the tie was broken
- **Response Assembly**: The poll data and computed results are returned to the client

Demo Example



References

- Zou, J., Meir, R., & Parkes, D. C. (2015). Strategic voting behavior in Doodle polls. In Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing (pp. 623–634). Association for Computing Machinery. https://doi.org/10.1145/2675133.2675213
- Freixas, J., & Pons, M. (2021). On anonymous and weighted voting systems. Theory and Decision, 91(4), 477–498. https://doi.org/10.1007/s11238-021-09814-3
- Huang, Z., Sydor, A., Macharis, C., et al. (2024). A collaborative group decision-support system: The survey based multi-actor multi-criteria analysis (MAMCA) software. Journal of the Operational Research Society, 75(5), 843–856. https://doi.org/10.1080/01605682.2023.2172727